
Happiness can be found,
even in the darkest of times,
if one only remembers
to turn on the light.
— ALBUS DUMBLEDORE —
HOGWARTS
COURAGE
FRIENDSHIP
KNOWLEDGE
KINDNESS
Happiness
can be found...
A BRIGHTER
TOMORROW
ALWAYS EXISTS

TLS 1.2

From Handshake to Key Separation

Lecture on ECDHE, RSA authentication, the pre-master/master-secret hierarchy, traffic keys, and the security reasoning behind the design.

TLS 1.2 | ECDHE | AES-GCM

Core question

Why does TLS derive multiple secrets instead of using the ECDHE shared secret directly?

Learning objectives

- Trace a TLS 1.2 ECDHE handshake message-by-message.
- Explain pre-master → master secret → key block → directional traffic keys.
- Reason about freshness, domain separation, purpose separation, direction separation, and forward secrecy.
- Contrast the TLS 1.2 key schedule with the cleaner TLS 1.3 approach.

MENTAL MODEL

Where TLS sits

Application

HTTP, SMTP, IMAP…

Security

TLS 1.2

Transport

TCP

Network

IP



TLS provides

Confidentiality • integrity/authenticity • peer authentication (usually server) • key establishment

BIG PICTURE

The four jobs of a TLS handshake

1. Negotiate

Version, cipher suite, extensions

2. Authenticate

Certificate + signature prove server identity

3. Establish keys

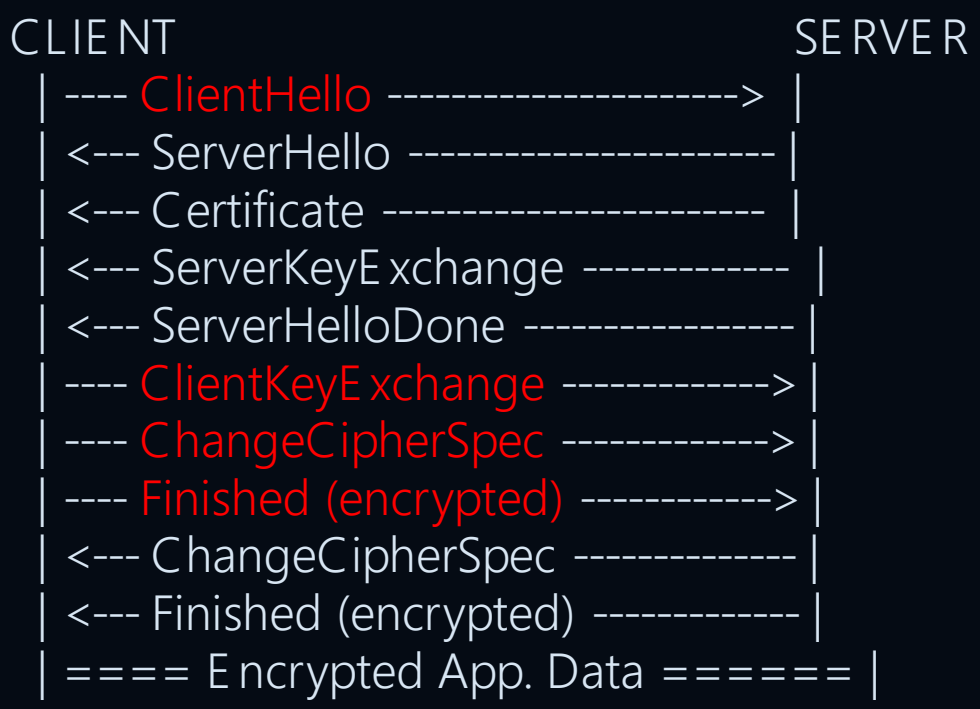
ECDHE creates shared secret

4. Confirm

Finished verifies secrets + transcript integrity

Then: protect application data with fast symmetric cryptography.

TLS 1.2 ECDHE handshake at a glance



NEGOTIATION BEGINS

Step 1 — ClientHello

Client offers

TLS version(s)
ClientRandom
Cipher suites
Extensions
Supported groups/curves
Signature algorithms

Security role

Advertises capabilities and contributes fresh handshake randomness. The ClientRandom later participates in secret derivation.

SERVER CHOOSES

Step 2 — ServerHello

Server selects

TLS 1.2
One cipher suite
ServerRandom
Negotiated extensions

```
TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
```

One name encodes most of the cryptographic plan.

ECDHE • RSA • AES-128-GCM • SHA-256

Decode the cipher suite

ECDHE

Ephemeral key agreement → shared secret

RSA

Server authentication via signature

AES-128-GCM

Authenticated encryption for TLS records

SHA-256

Hash used by TLS 1.2 PRF for this suite

Common trap: RSA is NOT the data-encryption algorithm here.

IDENTITY CLAIM

Step 3 — Certificate

Certificate contains

Subject / SAN names
Server RSA public key
Issuer
Validity period
CA signature
Other constraints

Client validates

Hostname matches
Chain reaches a trusted root
Signatures verify
Certificate is valid for intended usage
Time validity / policy checks

AUTHENTICATE THE EPHEMERAL KEY

Step 4 — ServerKeyExchange

Server creates

Temporary ECDHE private key
Temporary ECDHE public key

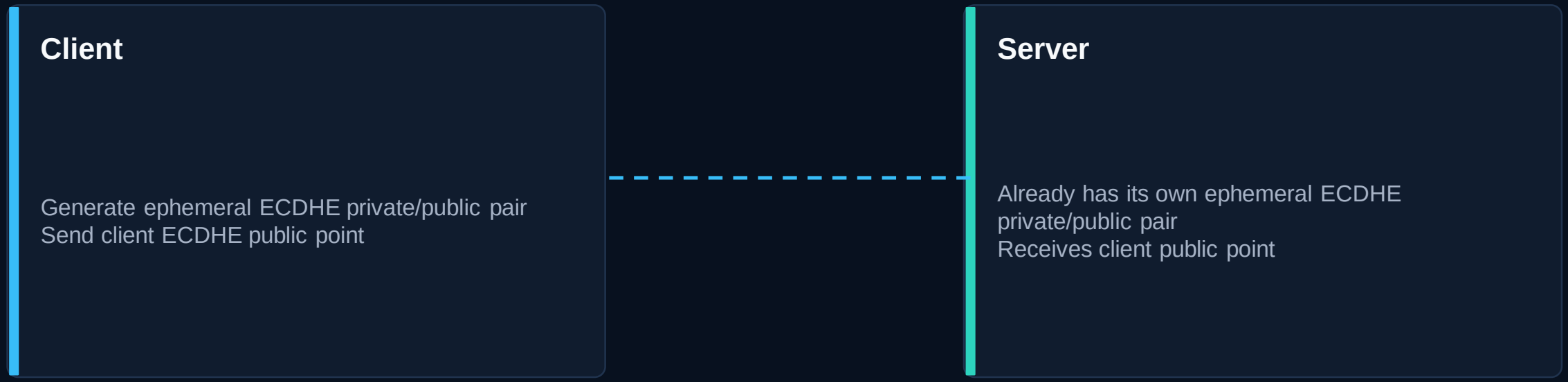
Server signs

ECDHE parameters + relevant handshake values are signed using the server's RSA private key.

The signature binds the ephemeral ECDHE parameters to the authenticated server.

CLIENT CONTRIBUTES ECDHE PUBLIC KEY

Step 5 — ClientKeyExchange



Private keys never cross the network.

KEY AGREEMENT

ECDHE — same secret, never transmitted

Client computes

client_private × server_public

→ ECDHE shared secret

Server computes

server_private × client_public

→ same ECDHE shared secret

An eavesdropper sees both public keys, but not the private scalars.

DESIGN QUESTION

Why not use the ECDHE output directly as AES key?

Tempting design:

```
AES_KEY = ECDHE_SHARED_SECRET
```

TLS instead builds a key hierarchy.

```
ECDHE → Pre-Master → Master
      → Key Block → Directional Keys
```

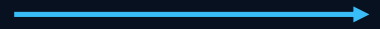
- Normalize/derive cryptographic key material.
- Bind derivation to this handshake's context.
- Separate purposes and traffic directions.
- Generate exactly the key/IV material the chosen cipher suite needs.

RAW SHARED KEYING MATERIAL

Pre-Master Secret

Source

For ECDHE suites: derived from the ECDHE shared secret according to TLS rules.



Role

Input key material to the TLS PRF. It is not the application-data AES key.

Think: “shared secret material before TLS-specific expansion.”

Pre-Master → Master Secret

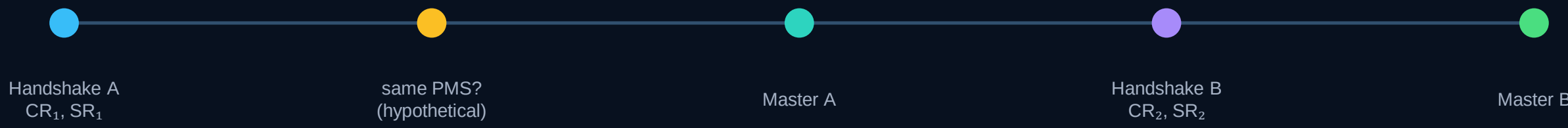
```
master_secret = PRF(
  pre_master_secret,
  "master secret",
  ClientRandom || ServerRandom
)

Length: 48 bytes
```

Security gains

- TLS-specific derivation
- Explicit purpose label
- Incorporates both handshake randoms
- Produces a fixed-size root session secret

Security gain #1 — context & freshness



Different handshake randoms → different PRF inputs → different derived master secrets (with overwhelming probability).

The randoms are public context, not passwords.

Security gain #2 — domain separation

Master-secret derivation

```
PRF(PMS,
    "master secret",
    CR || SR)
```

Key expansion

```
PRF(Master,
    "key expansion",
    SR || CR)
```

Different labels make different derivation domains.

A SUBTLE TLS 1.2 DETAIL

Why reverse CR||SR to SR||CR?

Master secret seed:
ClientRandom || ServerRandom

Key expansion seed:
ServerRandom || ClientRandom

Correct interpretation

The reversal distinguishes the derivation inputs, but the explicit labels (“master secret” vs “key expansion”) are the clearer domain-separation mechanism.

“TLS is secure because the randoms are reversed?”

No

EXPAND ONCE, PARTITION BY PURPOSE

Master Secret → Key Block

```
key_block = PRF(
  master_secret,
  "key expansion",
  ServerRandom || ClientRandom
)
```

AES-GCM suite needs

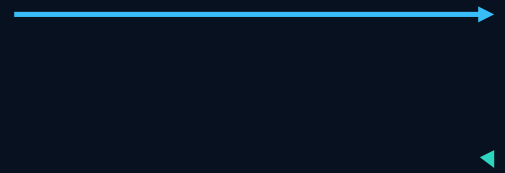
client_write_key
server_write_key
client_write_IV (fixed IV material)
server_write_IV (fixed IV material)

Older non-AEAD suites can also derive separate client/server MAC keys.

What “write key” actually means

Client Write Key

Client encrypts/authenticates outbound records. Server uses the same directional key to decrypt/verify them.



Server Write Key

Server encrypts/authenticates outbound records. Client uses it to decrypt/verify them.

“Write” names the sender/direction, not a different encryption vs decryption algorithm.

KC ≠ KS

Security gain #3 — direction separation

Client → Server

Kc = client_write_key
Client nonce/IV context
Independent sequence-number context

Server → Client

Ks = server_write_key
Server nonce/IV context
Independent sequence-number context

Ciphertext valid under one directional context is not automatically valid under the other.

ROOT SECRET IS NOT A TRAFFIC KEY

Security gain #4 — purpose separation



Root secret

Master secret stays conceptually above record protection.

Derived keys

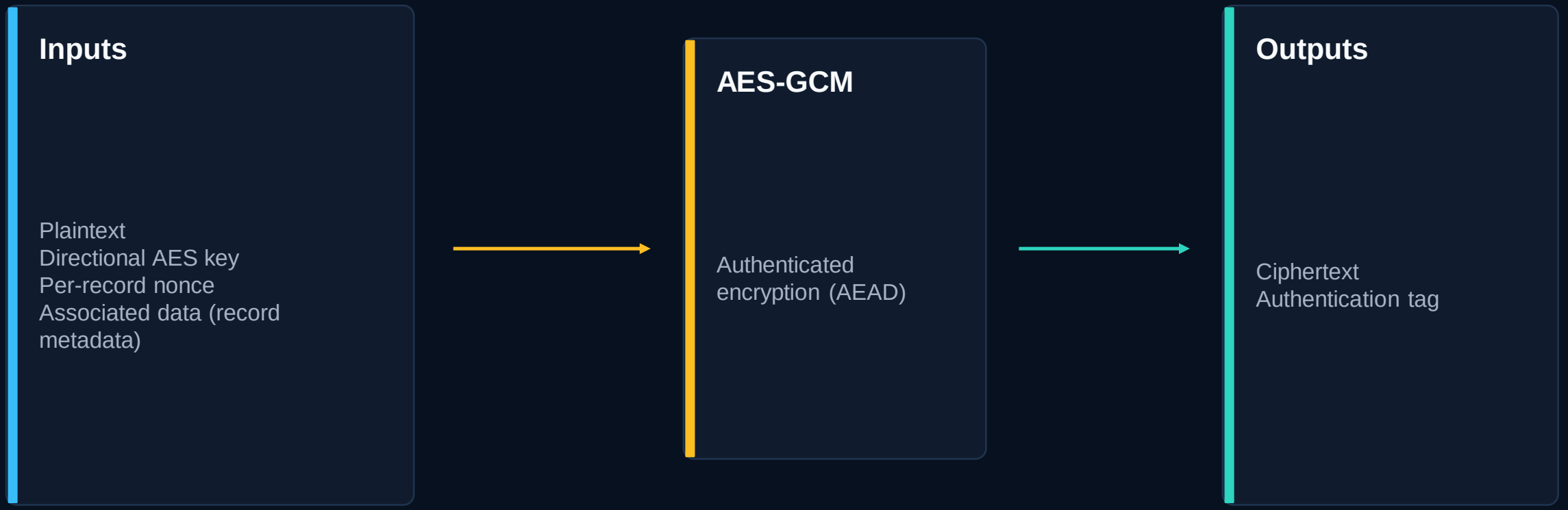
Use only for defined direction/purpose.

Crypto hygiene

Compartmentalize roles; minimize key reuse.

AFTER THE HANDSHAKE

AES-GCM record protection



GCM protects confidentiality AND detects modification.

Finished messages

Client Finished

Computed from the master secret and a hash of prior handshake messages; sent after client switches cipher state.

Server Finished

Server independently computes/verifies its Finished value, then sends its own.

Finished detects handshake tampering and confirms possession of the negotiated secret state.

Forward secrecy



Long-term RSA compromise does not, by itself, reconstruct old ECDHE shared secrets.

CAN YOU MAP EACH PRIMITIVE TO ITS JOB?

Knowledge check #1

TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256

ECDHE

?

RSA

?

AES-GCM

?

SHA-256

?

Ask learners before revealing: key agreement? authentication? record protection? PRF hash?

Knowledge check #2

ECDHE shared secret
↓
Pre-Master Secret
↓ PRF + "master secret" + CR||SR
Master Secret
↓ PRF + "key expansion" + SR||CR
Key Block
↓
Client Write Key / Server Write Key / IV material

Question: What security/design gain appears at each arrow?

FIX THESE BEFORE THEY STICK

Common misconceptions

“RSA encrypts the web traffic.”

Not in ECDHE_RSA. RSA authenticates; AES-GCM protects records.

“The randoms are secret.”

No. They are public handshake context used in derivation.

“SHA-256 MACs every GCM record.”

No. GCM already authenticates records; SHA-256 serves the TLS 1.2 PRF here.

“Client write key is only on client.”

Both endpoints need it; the name identifies the sending direction.

“Reversing CR/SR is the main security trick.”

No. Labels are the clearer domain separator; reversal is additional protocol distinction.

EVOLUTION

TLS 1.2 vs TLS 1.3 key schedule

TLS 1.2

Pre-Master Secret
↓
Master Secret
↓
Key Block
↓
Client/Server write keys

PRF-based schedule tied to cipher-suite design.

TLS 1.3

Early Secret
↓
Handshake Secret
↓
Master Secret
↓
Client/Server traffic secrets

HKDF with more explicit staged secret derivation.

SYNTHESIS

The complete security story



- Certificate + RSA signature: “Who are you, and can you prove it?”
- ECDHE: “Can we establish a shared secret without transmitting it?”
- PRF hierarchy: “Can we derive handshake-specific, purpose-specific keying material?”
- Client/server write keys: “Can we isolate the two traffic directions?”
- AES-GCM: “Can we efficiently encrypt and authenticate every record?”

Recap

Protocol flow

ClientHello → ServerHello → Certificate → ServerKeyExchange → ClientKeyExchange → CCS/Finished → protected data

Key hierarchy

ECDHE → Pre-Master → Master → Key Block → client/server write keys + IV material

Security principles

Authentication
Forward secrecy
Context binding
Domain separation
Purpose separation
Direction separation

Discussion prompts

- Why is a certificate insufficient without proof of private-key possession?
- What would go wrong if the same traffic key and nonce space were reused in both directions?
- What security property would be lost by replacing ECDHE with TLS 1.2 RSA key exchange?
- Why are public randoms still useful inputs to a key derivation function?
- Which TLS 1.2 concepts become cleaner or more explicit in TLS 1.3?